to 'True'. So, for example, *getent passwd user1* will not find the user while *getent passwd user1@DOMAIN* will.

Important file locations for SSSD:

- Configuration file: */etc/sssd/sssd.conf* (sample can be found at */usr/share/doc/sssd-1.8.90/sssd-example.conf*)
- Cache location: */var/lib/sss/db/*
- Log files directory: */var/log/sssd/*

Installation for Fedora-based systems (if not already installed) is as simple as:

```
# yum install sssd sssd-tools
```

Case 1: Replacement for local accounts

Local domain stores all user names and passwords internally in SSSD native database in the form of ldb (light-weight embedded database library). You will not be able to find these users and groups in */etc/passwd* or */etc/groups*, since those have been specifically left for *root* and *system* accounts. Note that the local domain is never offline, so the caching of user passwords is not done here. A basic configuration of */etc/sssd/sssd.conf* for the local domain would be as follows:

```
[sssd]
config_file_version = 2
services = nss, pam
domains = LOCAL

[nss]
filter_groups = root
filter_users = root

[pam]
reconnection_retries = 3

[domain/LOCAL]
description = Local users domain
id_provider = local
enumerate = true
min_id = 1001
max_id = 2000
```

For more directives and fine-tuning, refer to the manual page for *sssd.conf*.

Now, to add a local account user with SSSD management tools, issue the following command:

```
# sss_useradd localuser1
```

sss_useradd - - help will provide more options. Changing a password is just like changing your local account password:

```
# passwd localuser1
```

Logging in should now be possible to the *localhost* using SSH or the desktop manager; you should be able to verify that there are no entries for *localuser1* in */etc/passwd* or */etc/shadow*. Test user access using SSH, as follows:

```
# ssh -l localuser1 localhost
```

Similarly, to add a local group, use the command given below:

```
# sss_groupadd group1
```

Other commands from *sssd-tools* are: *sss_userdel, sss_cache, sss_debuglevel, sss_groupdel, sss_groupshow, sss_usermod, sss_groupmod and sss_obfuscate*.

Case 2: Network accounts with the LDAP back-end

This back-end requires an active LDAP server to authenticate against. Note that TLS/SSL is mandatory to authenticate against the server. SSSD does not support authentication over unencrypted channels—however, if the server is used just as an identity provider, an encrypted channel is not required. The identity provider supports three schemas—the RFC2307, RFC2307bis and IPA schema. RFC2307 is the default. Here's a sample configuration of sssd.conf for the native LDAP back-end:

```
[sssd]
config_file_version = 2
services = nss, pam
domains = LDAP

[nss]
filter_groups = root
filter_users = root

[pam]
reconnection_retries = 3
```

Frequently asked questions

1. *I see old user information; so how do I clear cache?*
   At times it becomes necessary to clear the cache information of certain users or groups instead of waiting for time-out. In such cases, you can use the sss_cache tool:

```
# sss_cache --user localuser1
```

This invalidates records in the SSSD cache; invalidated records are forced to be reloaded from the server.

2. *How do I set a level for debugging?*
   You can change the debug level on-the-fly without having to specify it in the configuration and reload the sssd daemon. This setting overrides the settings from the config file:

```
# sss_debuglevel 9
```